DevOps – Lab Program-2 Documentation

Title:Develop a Multi-Stage Dockerfile for Node.js Application

Objective:

To build and deploy a Node.js web application using a multi-stage Dockerfile, optimizing image size and enabling efficient container orchestration.

Software Requirements:

- Docker
- Node.js
- Express.js
- VS Code / Terminal

Files and Folder Structure:

```
SecondProgram/
│
├── node_modules/
├── src/
│   └── index.js
├── package.json
├── package-lock.json
└── Dockerfile
```

Source Code:

index.js

```
const express = require('express');
const app = express();
```

```
const port = 2000;
app.get('/', (req, res) => {
res.send(`
<script>alert("Hello from the docker file")</script>
`);
});app.listen(port, () => {
console.log("running");
});
```

package.json

```
{
"name": "secondprogram",
"version": "1.0.0",
"main": "index.js",
"scripts": {
"build": "echo \"Building the project...\" && mkdir -p dist && cp -r src/*
dist/",
"start": "node dist/index.js",
"test": "echo \"Error: no test specified\" && exit 1"
},
"dependencies": {
"express": "^5.1.0"
}
}
```

Dockerfile

```
# Stage 1: Build stage
FROM node:20-alpine AS builder
WORKDIR /app
COPY package.json ./
RUN npm install
COPY ./src ./src
RUN npm run build
# Stage 2: Production stage
FROM node:20-alpine
```

```
WORKDIR /app
COPY --from=builder /app/package.json ./
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist
EXPOSE 2000
CMD ["npm", "start"]
```

Procedure:

1. Initialize Project:
mkdir SecondProgram
cd SecondProgram
2. Build Docker Image:
sudo docker build -t up .
3. Run Docker Container:
sudo docker run -d -p 2000:2000 up
4. Open Browser and Visit:
http://localhost:2000
5. You will see the alert message:
"Hello from the docker file"

OUTPUT AND SCREENSHOTS:

src > JS index.js > ...

```
1   const express = require('express');
2   const app = express();
3   const port = 2000;
4   app.get('/', (req, res) => {
5   res.send(`
6   <script>alert("Hello from the docker file")</script>
7   `);
8   });app.listen(port, () => {
9   console.log("running");
10  });
11
```

{} package.json × {} package-lock.json JS index.js Dockerfile

{} package.json > {} scripts

```
1   {
2     "name": "secondprogram",
3     "version": "1.0.0",
4     "main": "index.js",
      ▷Debug
5     "scripts": {
6       "build": "echo \"Building the project...\" && mkdir -p dist && cp -r src/*
7       "start":" node dist/index.js",
8       "test": "echo \"Error: no test specified\" && exit 1"
9     },
10    "keywords": [],
11    "author": "",
12    "license": "ISC",
13    "description": "",
14    "dependencies": {
15      "express": "^5.1.0"
16    }
17  }
```

package.json | Dockerfile | package-lock.json | index.js

Dockerfile

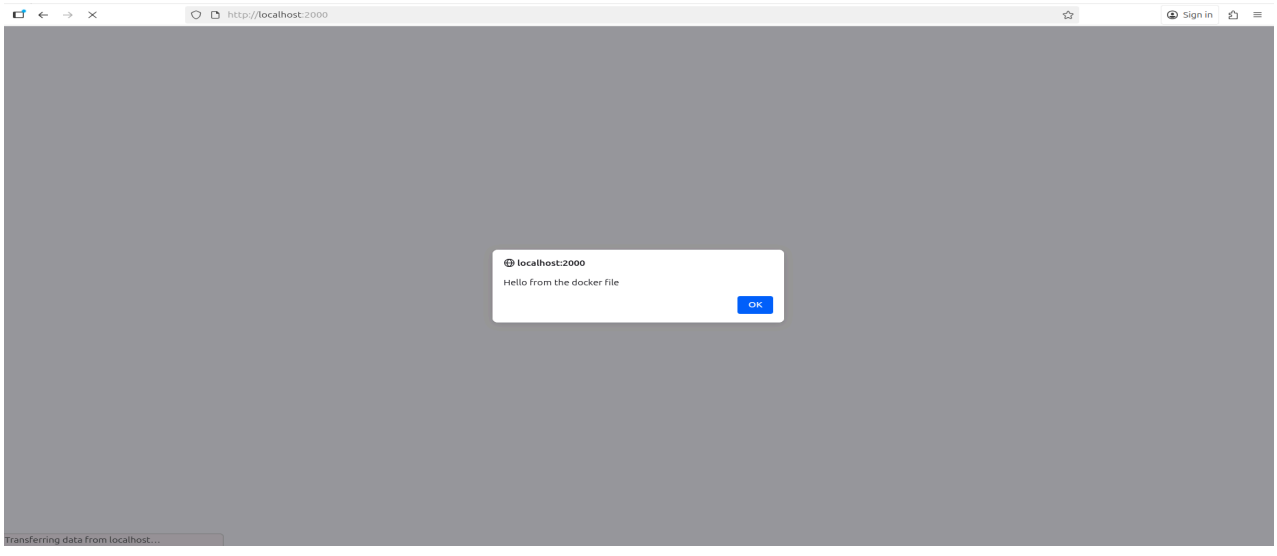
```
# Stage 1: Build stage
FROM node:20-alpine AS builder
WORKDIR /app
COPY package.json ./
RUN npm install
COPY ./src ./src
RUN npm run build
# Stage 2: Production stage
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/package.json ./
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist
EXPOSE 2000
CMD ["npm", "start"]

# End of Dockerfile
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

bash

```
1rv24mc006_adityamukherjee@asus-ASUS-TUF-Dash-F15-FX516PR-FX516PR:~/seco$ sudo docker build -t pp2 .
 => => transferring dockerfile: 423B                                                                                    0.0s
 => [internal] load metadata for docker.io/library/node:20-alpine                                                       7.9s
 => [internal] load .dockerignore                                                                                       0.0s
 => => transferring context: 2B                                                                                         0.0s
 => [internal] load build context                                                                                       0.0s
 => => transferring context: 727B                                                                                       0.0s
 => [builder 1/6] FROM docker.io/library/node:20-alpine@sha256:6178e78b972f79c335df281f4b7674a2d85071aae2af020ffa39f0a770265435   0.0s
 => CACHED [builder 2/6] WORKDIR /app                                                                                   0.0s
 => [builder 3/6] COPY package.json ./                                                                                  0.0s
 => [builder 4/6] RUN npm install                                                                                       3.8s
 => [builder 5/6] COPY ./src ./src                                                                                      0.0s
 => [builder 6/6] RUN npm run build                                                                                     0.2s
 => [stage-1 3/5] COPY --from=builder /app/package.json ./                                                              0.0s
 => [stage-1 4/5] COPY --from=builder /app/node_modules ./node_modules                                                  0.1s
 => [stage-1 5/5] COPY --from=builder /app/dist ./dist                                                                  0.0s
 => exporting to image                                                                                                  0.1s
 => => exporting layers                                                                                                 0.1s
 => => writing image sha256:a07123e7028846e6cdfa2f3016e6e8cdc676a4aaab133fea1b07af1b2b1c5fa6                            0.0s
 => => naming to docker.io/library/pp2                                                                                  0.0s
1rv24mc006_adityamukherjee@asus-ASUS-TUF-Dash-F15-FX516PR-FX516PR:~/seco$
```



Result:

Successfully developed a multi-stage Dockerfile that builds and runs a Node.js application on port 2000 using Express.js.

Conclusion:

This experiment demonstrates the use of multi-stage Docker builds for creating optimized, lightweight, and production-ready container images.